QP – 1

📘 **Algorithm Design Techniques – Model Question Paper**

Total Marks: 100
Duration: 3 Hours

🔹 **PART A – Short Answer (10 × 2 = 20 Marks)**

Answer **all** questions.

1. Define **Big O notation**. What does it represent in algorithm analysis?
2. What are the **two key properties** required for a problem to be solvable using **Dynamic Programming**?
3. State the **core principle** of a **Greedy algorithm**.
4. What is the **main difference** between **Prim's** and **Kruskal's** algorithms for finding a **Minimum Spanning Tree (MST)**?
5. Define **Backtracking** as a problem-solving technique.
6. How does the **Rabin-Karp algorithm** use hashing for string matching?
7. What are the **Bad Character Heuristic** and **Good Suffix Heuristic** in the **Boyer-Moore algorithm**?
8. State the **constraints** in the **N-Queen problem**.
9. What is the **advantage** of using a **Segment Tree** over brute-force for **Range Minimum Queries (RMQ)**?
10. Differentiate between the **0-1 Knapsack** and **Fractional Knapsack** problems.

🔹 **PART B – Long Answer (5 × 16 = 80 Marks)**

Answer **all** questions, choosing either (a) or (b) for each.

1.

(a)

i. Explain the **Activity Selection Problem**. Write the **Greedy algorithm** to find the maximum number of compatible activities. (8)

ii. Solve the recurrence relation using **Master Theorem**:
$T(n) = 3T(n/2) + n^2$ (8)

OR

(b)

i. Write the algorithm for the **Fractional Knapsack Problem**. (8)

ii. Prove the **optimality** of the greedy strategy used. (8)

2.

(a)
Explain the complete **Knuth-Morris-Pratt (KMP)** algorithm for string matching. Include:

- How to compute the **prefix function (LPS array)**
- The matching algorithm
- Trace the algorithm to find pattern **"AABA"** in text **"AABAACAADAABAABA"** (16)

OR

(b)

i. Write the recurrence relation for **Longest Common Subsequence (LCS)**. (6)

ii. Use **Dynamic Programming** to find the LCS of:
X = "AGGTAB", Y = "GXTXAYB"
Show the filled **DP table**. (10)

3.

(a)
Design a **Backtracking algorithm** to solve a **Sudoku puzzle**. Explain:

- State-space
- Branching logic
- Constraints used for pruning (16)

OR

(b)

i. Describe the **Graph M-Coloring Problem**. (6)

ii. Write the **Backtracking algorithm** to find all possible solutions for coloring a graph with **m colors**. (10)

4.

(a)
Formulate the **0-1 Knapsack Problem** using **Dynamic Programming**.

- Explain the recurrence relation
- Write the algorithm
- Trace it for knapsack capacity **W = 5** with items:
  Item 1: Weight = 2, Value = 6
  Item 2: Weight = 2, Value = 10
  Item 3: Weight = 3, Value = 12 (16)

OR

(b)

i. Explain the **Longest Increasing Subsequence (LIS)** problem. (6)

ii. Write the **$O(n^2)$** DP algorithm to solve LIS.
Trace it on: **{10, 22, 9, 33, 21, 50, 41, 60}** (10)

5.

(a)
Explain **Dijkstra's algorithm** for **Single-Source Shortest Path**.

- Trace it on a weighted, directed graph with **at least 5 nodes**
- State its **time complexity** using a **binary heap** (16)

OR

(b)

i. Explain the **Floyd-Warshall algorithm** for **All-Pairs Shortest Paths**.
State its **time complexity** and reasoning. (8)

ii. What is a **Segment Tree**?
Explain its construction and usage for **sum queries** in range **[i, j]**. (8)

QP – 2

Algorithm Design Techniques (UG Level)

Maximum Marks: 100 **Time:** 3 Hours
Note:

- Answer **ALL questions** in Section A.
- Answer **ANY FIVE** questions from Section B (Each question carries 16 marks).
- In Section B, **11 OR 12** is an internal choice.

Section A (10 × 2 = 20 Marks)

(Answer all questions. Each question carries 2 marks.)

1. Define **time complexity** and give an example of an algorithm with (O(n \log n)) complexity.
2. What is the difference between **greedy algorithms** and **dynamic programming**?
3. Write one real-life example where **pattern matching algorithms** are applied.
4. What is a **heap**? Mention one use case of a **priority queue**.
5. State the **principle of optimality** used in dynamic programming.
6. Write the base condition for the **0-1 Knapsack problem** in dynamic programming.
7. What is the **purpose of backtracking** in problem-solving?
8. Differentiate between **Prim's** and **Kruskal's** algorithms.
9. What is the **time complexity** of building a **segment tree**?
10. Define **Longest Common Subsequence (LCS)** with an example.

Section B (5 × 16 = 80 Marks)

(Answer any FIVE questions. Question 11 OR 12 carries internal choice.)

Q.11. (a) Explain the **Divide and Conquer** technique with an example. Derive its recurrence relation and solve it using the **Master Theorem**. *(10 Marks)*

(b) Apply the **Master Theorem** to solve the following recurrence:
($T(n) = 3T(n/2) + n \log n$). *(6 Marks)*

OR

Q.12. (a) Design a **greedy algorithm** for the **Fractional Knapsack problem**. Explain the correctness proof and its time complexity. *(10 Marks)*
(b) Differentiate between **Fractional Knapsack** and **0/1 Knapsack** problems. *(6 Marks)*

Q.13.
(a) Describe the **Rabin-Karp algorithm** for string matching. What is its worst-case complexity? *(8 Marks)*
(b) Compare Rabin-Karp with **Naïve** and **KMP** algorithms. *(8 Marks)*

Q.14.
(a) Explain how the **Subset Sum Problem** is solved using **dynamic programming**. Include the recursive formulation and bottom-up approach. *(10 Marks)*
(b) Write pseudocode for **Longest Increasing Subsequence** using DP and discuss its time complexity. *(6 Marks)*

Q.15.
(a) Explain the **Backtracking approach** to solve the **N-Queens Problem**. Show the solution tree for ($N = 4$). *(10 Marks)*
(b) Discuss how **Backtracking** can be used for the **Sudoku Solver**. *(6 Marks)*

Q.16.
(a) Discuss **Dijkstra's Algorithm** for finding the shortest path in a graph. Explain with an example. *(10 Marks)*
(b) Write a short note on **Floyd–Warshall Algorithm** and its applications. *(6 Marks)*

Q.17.
(a) Explain the **Range Minimum Query (RMQ)** problem and its segment tree implementation. *(10 Marks)*
(b) How does the **Sparse Table** approach improve query time compared to segment trees? *(6 Marks)*

QP – 3

Algorithm Design Techniques

Total Marks: 100
Time: 3 Hours
Pattern: 10 × 2 Marks + 5 × 16 Marks (with internal choices)

PART A — Short Answer Questions

(Answer ALL questions, each question carries 2 marks.)

1. Define dynamic programming and give one example where it is used.
2. What is the time complexity of binary search? Justify your answer.
3. Explain the concept of greedy choice property in algorithms with an example.
4. What is the difference between a recursive and an iterative approach?
5. State the principle of optimality in dynamic programming.
6. Write the recurrence relation for the Tower of Hanoi problem.
7. What is a min-heap? List two of its applications.
8. Briefly describe the Sieve of Eratosthenes algorithm.
9. What is the importance of the Z algorithm in string matching?
10. List two differences between Kruskal's and Prim's algorithms.

PART B — Long Answer Questions

(Answer any FIVE. Each question carries 16 marks. Each question has an option — attempt either (a) or (b).)

11.

(a) Describe the working of the Rabin-Karp string matching algorithm and analyse its best and worst case time complexity.
OR
(b) With a neat diagram and suitable example, explain the working of the activity selection problem using a greedy approach.

12.

(a) Solve the following using dynamic programming: Given a set of coins with denominations 1, 3, and 4, find the minimum number of coins required to make a change for 7. Write the steps clearly.
OR
(b) Explain the subset sum problem and provide a pseudo-code for its solution using recursion and memoization.

13.

(a) Given a connected, undirected graph, explain Dijkstra's algorithm for finding the shortest path from a source node. Show all steps for a sample graph.
OR
(b) What is the Floyd-Warshall algorithm? Illustrate its use for finding all pairs shortest paths in the following graph (provide a small 4-node graph in your answer).

14.

(a) State and solve the N-Queen problem for N=4 using backtracking. Show the state space tree and all solutions.
OR
(b) Explain the Knight's Tour problem using backtracking. Provide the steps for a 5 × 5 chessboard.

15.

(a) Compare and contrast the greedy, divide-and-conquer, and dynamic programming approaches with the help of suitable examples for each.
OR
(b) Given an array of integers, explain how you would find the maximum subarray sum (Kadane's algorithm). Write the algorithm and discuss its time complexity.

QP – 4

Algorithm Design Techniques – Question Paper

Total Marks: 100
Pattern: 10 × 2 marks + 5 × 16 marks (with internal choice)
Difficulty: Mixed (Easy, Medium, Hard)

Part A – Short Answer Questions (10 × 2 = 20 Marks)

Answer all questions. Each question carries 2 marks.

1. Define Time Complexity. Give an example of a recursive algorithm and its time complexity. *(Easy)*
2. What is a Min-Heap? How does it differ from a Max-Heap? *(Easy)*
3. State the principle behind the Activity Selection Problem. *(Medium)*
4. Explain the role of hashing in Rabin-Karp algorithm. *(Medium)*
5. What is the significance of the Z-array in Z algorithm? *(Medium)*
6. Write the base case and recursive case for solving the Tower of Hanoi problem. *(Medium)*
7. What is the difference between Top-down and Bottom-up approaches in Dynamic Programming? *(Easy)*
8. Mention two real-life applications of KMP algorithm. *(Easy)*
9. What is the purpose of Union-Find in graph algorithms? *(Hard)*
10. Compare Segment Tree and Sparse Table in terms of preprocessing and query time. *(Hard)*

Part B – Long Answer Questions (5 × 16 = 80 Marks)

Answer any five questions. Each question carries 16 marks. Internal choice is provided.

Q11. (Time Complexity & Recursion) ***(Medium-Hard)***

(a) Perform mathematical analysis of the recursive algorithm for computing GCD.
(b) Derive the time complexity of the Sieve of Sundaram algorithm.

OR
(a) Explain the recursive solution for the Palindrome Check problem.
(b) Analyze the time complexity of the Tower of Hanoi problem.

Q12. (Greedy Algorithms & Heaps) ***(Medium)***

(a) Implement Heap Sort using a Binary Heap and explain its time complexity.
(b) Solve the Fractional Knapsack problem using Greedy strategy.

OR
(a) Describe the Activity Selection Problem and solve it using Greedy approach.
(b) Explain the Graph Coloring Problem and its greedy solution.

Q13. (String Algorithms) ***(Medium-Hard)***

(a) Compare Naive, Rabin-Karp, and Z algorithms for pattern matching.
(b) Explain the working of Manacher's algorithm with an example.

OR
(a) Describe the KMP algorithm and its preprocessing phase.
(b) Explain the Boyer-Moore algorithm and its heuristics.

Q14. (Backtracking & Dynamic Programming) ***(Hard)***

(a) Solve the N-Queen problem using Backtracking.
(b) Discuss the Subset Sum problem and solve it using Dynamic Programming.

OR
(a) Explain the Knight's Tour problem and its backtracking solution.
(b) Compare Greedy and DP approaches for solving the Longest Increasing Subsequence problem.

Q15. (Graph & Range Query Algorithms) ***(Hard)***

(a) Implement Dijkstra's algorithm and analyze its time complexity.
(b) Construct a Segment Tree for Range Minimum Query and perform a query.

OR
(a) Explain Floyd-Warshall algorithm for All-Pairs Shortest Path.
(b) Compare Segment Tree and Sparse Table for Range Sum Query with examples.

QP – 5

Algorithm Design Techniques – Question Paper

Total Marks: 100
Pattern: 10 × 2 marks + 5 × 16 marks (with internal choice)

Part A – Short Answer Questions (10 × 2 = 20 Marks)

Answer all questions. Each question carries 2 marks.

1. Derive the recurrence relation for the time complexity of merge sort and solve it using the Master theorem. *(Hard)*

2. Explain how the choice of pivot affects the time complexity of quicksort in the worst case. Provide mathematical justification. *(Hard)*
3. In the context of heap operations, derive the time complexity for building a heap from an unsorted array using the bottom-up approach. *(Hard)*
4. Why does the Rabin-Karp algorithm use rolling hash? Analyze the probability of false positives with multiple pattern matches. *(Hard)*
5. Prove that the Z-algorithm runs in O(n) time complexity despite having nested loops. *(Hard)*
6. What is the space complexity of the recursive Tower of Hanoi solution? Justify your answer with the call stack analysis. *(Medium)*
7. Explain the concept of overlapping subproblems with an example where both greedy and dynamic programming approaches can be applied but yield different results. *(Hard)*
8. Derive the failure function for the pattern "ABABCABAA" in the KMP algorithm. *(Medium)*
9. Explain the path compression and union by rank optimizations in Union-Find. How do they affect the amortized time complexity? *(Hard)*
10. Why is a sparse table preferred over a segment tree for static range queries? Provide a time-space complexity trade-off analysis. *(Hard)*

Part B – Long Answer Questions (5 × 16 = 80 Marks)

Answer any five questions. Each question carries 16 marks. Internal choice is provided.

Q11. Advanced Time Complexity & Recursion Analysis *(Hard)*

(a) Analyze the time complexity of the recursive algorithm $T(n) = 2T(n/2) + n \log n$ using the recursion tree method. Verify using the Master theorem. Explain why the Master theorem applies or doesn't apply.

(b)

(b) Implement the Sieve of Sundaram algorithm and prove mathematically why it generates all odd primes up to n. Compare its efficiency with the Sieve of Eratosthenes in terms of space and time complexity.

OR

(c) Solve the recurrence relation for the Fibonacci sequence using the characteristic equation method. Explain why memoization reduces the time complexity from $O(2^n)$ to O(n).

(d)

(b) Derive the exact number of moves required for the Tower of Hanoi with 4

pegs (Reve's puzzle) and prove that it's optimal. Compare with the 3-peg solution.

Q12. Advanced Heap Operations & Greedy Algorithm Optimization *(Hard)*

(e) Implement a k-way merge using a min-heap for merging k sorted arrays. Analyze the time complexity and prove its optimality. Discuss how this applies to external sorting.

(f)

(b) Solve the weighted job scheduling problem using dynamic programming. Prove why a greedy approach fails for this variant.

OR

(g) Design an algorithm to find the median in a stream of integers using two heaps. Analyze the time complexity for insertion and median retrieval.

(h)

(b) Solve the graph coloring problem using backtracking with pruning optimizations. Prove the NP-completeness of the decision version and discuss approximation algorithms.

Q13. Advanced String Pattern Matching & Algorithm Engineering *(Hard)*

(i) Implement the Z-algorithm for pattern matching and prove its O(n + m) time complexity using the concept of Z-boxes. Explain how it can be used to solve the string rotation problem efficiently.

(j)

(b) Analyze Manacher's algorithm for finding the longest palindromic substring. Prove why it achieves linear time complexity. Implement it with detailed expansion logic.

OR

(k) Compare the Boyer-Moore algorithm's bad character and good suffix heuristics. Prove the worst-case and best-case time complexities. Show an example where Boyer-Moore outperforms KMP significantly.

(l)

(b) Design an algorithm to find all occurrences of an anagram of a pattern in a text using rolling hash. Analyze the expected and worst-case time complexities. *(Medium)*

Q14. Complex Backtracking & DP with Optimization *(Hard)*

(m) Solve the Hamiltonian cycle problem using backtracking with intelligent pruning. Prove its exponential time complexity and discuss why no polynomial-time solution exists. Implement with optimization techniques.

(n)

(b) Solve the matrix chain multiplication problem using dynamic programming. Derive the optimal substructure and prove correctness. Reconstruct the optimal parenthesization.

OR

(o) Implement a Sudoku solver using backtracking with constraint propagation and naked singles technique. Analyze why these optimizations reduce the search space.

(p)

(b) Solve the longest common subsequence (LCS) problem for three strings using 3D dynamic programming. Prove the optimal substructure and analyze space optimization techniques.

Q15. Advanced Graph Algorithms & Complex Range Queries *(Hard)*

(q) Implement Dijkstra's algorithm using a Fibonacci heap and prove its O(E + V log V) time complexity. Compare with binary heap implementation. Extend to solve the k-shortest paths problem.

(r)

(b) Design a segment tree with lazy propagation to handle range update and range sum queries. Prove the time complexity and implement a solution for the range update query problem.

OR

(s) Implement the Bellman-Ford algorithm and prove its correctness for detecting negative cycles. Compare with Floyd-Warshall for dense vs sparse graphs. Discuss Johnson's algorithm for all-pairs shortest paths with negative edges.

(t)

(b) Construct a persistent segment tree to answer range queries on historical versions of an array. Analyze the space-time trade-off. Solve a problem requiring queries like "what was the sum of range [L, R] after the kth update?"
